

DEV-590: Create service-specific Redis user isolation guide

Department: DevOps

Prepared by: Thanusha Bai V

Date: 17-sept-2026

Task: DEV-587-Redis Security Hardening

1. Executive Summary

This guide defines how Glynac should isolate Redis access per service using Access Control Lists (ACLs). Instead of sharing one privileged connection, each service authenticates as its own user with the minimum set of keys, commands, and channels it needs. The design was validated live on a reference Redis 8.10.1 instance – five users were created, persistence via `aclfile` was confirmed across a restart, and isolation was proven with seven functional tests. All tests passed. A critical pitfall discovered during setup (locking yourself out by disabling default while connected as it) is documented with a safe rollout procedure.

2. Design Principle: Least Privilege

Each service gets:

- Its own **username + strong password** (32+ chars)
 - A **scoped key pattern** (not `~*`)
 - A **minimal command set** – no `@all`, no `@dangerous`
 - **Channels reset** (`resetchannels`) unless Pub/Sub is required
 - The **default user disabled** entirely
-

3. Glynac Service User Model

Service	Key pattern	Commands granted	Purpose
webapp	~session:* ~cache:*	+get +set +del +expire +ttl +exists	Session store + page cache
worker	~jobs:*	+rpush +lpop +blpop +brpop +lrange +llen +del	Background job queue
monitor	~*	+@read -keys	Read-only dashboards/met rics
admin	~* &*	+@all	Operations only – never used by apps
default	–	–	Disabled

Every non-admin user is created with `-@dangerous` and `resetchannels` to deny dangerous commands and Pub/Sub by default.

4. Live Configuration Commands

Run as an existing admin (or initially as default before disabling it):

```
ACL SETUSER admin on >AdminPass_ChangeMe_32chars! ~* &*
+@all
ACL SETUSER webapp on >WebAppPass_ChangeMe_32chars! ~session:*
~cache:* +GET +SET +DEL +EXPIRE +TTL +EXISTS -@dangerous
ACL SETUSER worker on >WorkerPass_ChangeMe_32chars! ~jobs:* +RPUSH
+LPOP +BLPOP +BRPOP +LRANGE +LLEN +DEL -@dangerous
ACL SETUSER monitor on >MonitorPass_ChangeMe_32chars! ~* +@read
-KEYS -@dangerous
ACL SETUSER default off
ACL SAVE
```

Requirement: `aclfile /etc/redis/users.acl` must be set in `redis.conf`, otherwise `ACL SAVE` fails and users are lost on restart.

5. Verified Configuration (Post-Restart)

- 1) "user admin on sanitize-payload #... ~* &* +@all"
- 2) "user default on nopass sanitize-payload ~* &* +@all" ← disabled after Step 4
- 3) "user monitor on sanitize-payload #... ~* resetchannels -@all +@read -keys -@dangerous"
- 4) "user webapp on sanitize-payload #... ~session:* ~cache:* resetchannels -@all +get +set +del +expire +ttl +exists -@dangerous"
- 5) "user worker on sanitize-payload #... ~jobs:* resetchannels -@all +rpush +lpop +blpop +brpop +lrange +llen +del -@dangerous"

Persistence proven: After `systemctl restart redis-server`, all users remained (because ACL changes were saved to `/etc/redis/users.acl`).

6. Isolation Test Results

#	Test	Expected	Result
1	webapp writes session:abc	✓ OK	✓ OK
2	webapp writes jobs:123 (not its key)	✗ Deny	✗ NOPERM No permissions to access a key
3	webapp runs CONFIG GET	✗ Deny	✗ NOPERM ... 'config get'
4	monitor reads session:abc	✓ OK	✓ "hello"
5	monitor writes a key	✗ Deny	✗ NOPERM ... 'set'
6	worker pushes to jobs:queue	✓ OK	✓ (integer) 1
7	Unauthenticated SET foo bar	✗ Deny	✗ NOAUTH Authentication required

All 7 tests behaved as expected. Key isolation, command isolation, and authentication enforcement all work.

7. Critical Pitfall – Lockout Recovery

During setup we accidentally ran ACL SETUSER default off while still connected as default. Result: the session immediately lost all permissions, NOAUTH errors on every subsequent command, and – because aclfile wasn't configured yet – a restart wiped all users, leaving **zero ways to authenticate**.

Safe rollout order (ALWAYS follow this)

1. Create admin user first.
2. Test admin login **from a separate terminal**: `redis-cli --user admin --pass '...' PING` → must return PONG.
3. Only then disable default.

4. Ensure `aclfile` is configured **before** making changes, so `ACL SAVE` works.

Recovery if locked out

```
sudo systemctl stop redis-server
sudo -u redis redis-server /etc/redis/redis.conf --aclfile ""
# In a new terminal, reconnect as default (now enabled), recreate users,
ACL SAVE
# Then Ctrl+C the manual Redis, and:
sudo systemctl start redis-server
```

8. Operational Rules

1. **Applications never use admin or default.** Each service uses only its own user.
2. **Passwords live in a secrets manager**, not in config files or code.
3. **Rotate passwords** by adding a second password (`ACL SETUSER webapp >newpass`), updating apps, then removing the old (`ACL SETUSER webapp <oldpass`).
4. **Persist after every change:** `ACL SAVE`.
5. **Review ACL LOG** periodically for denied-command attempts.
6. **Never share usernames** across services – one user per service, always.

9. Summary

Area	Outcome
Design	5 users: admin, webapp, worker, monitor, default(off)
Key isolation	✅ Verified (Tests 1-2, 4, 6)
Command isolation	✅ Verified (Tests 3, 5)
Auth enforcement	✅ Verified (Test 7)
Persistence	✅ Verified across restart via aclfile
Lockout pitfall	⚠️ Documented with safe rollout procedure

10. Recommendations for Glynac

1. Adopt this 5-user model as the **baseline ACL template** for every Redis instance.
2. Store passwords in Glynac's secrets manager (Vault/AWS Secrets Manager/etc.).
3. Configure aclfile before any ACL changes – non-negotiable.
4. Apply the safe rollout order in Section 7 during production rollouts.
5. Extend the model per service as Glynac's service list grows.

*This guide is validated against a live Redis 8.10.1 instance.
Tested results are captured in Sections 5 and 6.*

END OF REPORT